

Parallelized Image Recognition System Using Multicore Programming (Project Report)

Gbemavo Opeyemi Yonlonfun
Department of Software Engineering
Univ. of Europe for Applied Sciences
Konrad-Ruse Ring 11, 14469 Potsdam, Germany.
gbemavo.yonlonfun@ue-germany.de

Prof. Mugdha Kashyap
Department of Software Engineering
Univ. of Europe for Applied Sciences
Konrad-Ruse Ring 11, 14469 Potsdam, Germany.
mugdha.kashyap@ue-germany.de

ABSTRACT

Image recognition systems are increasingly employed in numerous application domains, including autonomous vehicles, healthcare, industrial automation, and intelligent surveillance systems. However, sequential image processing approaches often experience significant computational delays when processing large datasets. This project presents a parallelized image recognition system implemented entirely in C++ using multicore programming techniques. The proposed framework utilizes OpenCV for image preprocessing and Histogram of Oriented Gradients (HOG) for feature extraction, while image classification is performed using the K-Nearest Neighbors (KNN) algorithm. OpenMP is employed to parallelize computationally intensive feature extraction operations across multiple CPU cores. Experimental evaluation was conducted using a dataset containing 4,020 images distributed among four classes: cat, dog, airplane, and automobile. Performance analysis demonstrated substantial improvements in execution efficiency, achieving a maximum speedup of 4.54x when executed using eight threads. The findings confirm that multicore parallelization effectively reduces execution time and improves scalability for image recognition applications.

I. INTRODUCTION

A. Background

Image recognition has emerged as one of the most important technologies in modern computing systems [1]. It is widely employed in numerous application domains, including autonomous vehicles, healthcare, industrial automation, robotics, intelligent transportation systems, and surveillance systems. Advances in computer vision and artificial intelligence have significantly improved the ability of computers to automatically analyze and interpret visual information. As the volume of image data generated daily continues to increase, efficient image processing techniques have become essential for real-time and large-scale applications. Traditional image recognition systems typically process images sequentially, where

images are analyzed one after another on a single processing core. Although sequential processing approaches are relatively straightforward to implement, they often suffer from high execution times and poor scalability when handling large datasets. These limitations become particularly significant in applications requiring real-time decision-making or processing thousands of images simultaneously. The rapid evolution of multicore processors provides an effective solution to these computational challenges. Modern processors contain multiple processing cores capable of executing tasks concurrently. By exploiting multicore architectures through parallel programming techniques, computationally intensive image processing tasks can be significantly accelerated. Multicore programming enables the distribution of workloads among multiple processor cores, thereby reducing execution time and improving overall system performance. OpenMP is a widely used Application Programming Interface (API) that supports shared-memory parallel programming [2], in C and C++. It provides a simple and efficient approach for parallelizing computational tasks on multicore systems. OpenMP enables developers to create parallel applications with minimal modifications to existing sequential code, making it particularly suitable for accelerating image processing algorithms. This project investigates the use of multicore programming techniques to develop a parallelized image recognition system implemented entirely in C++. The system employs OpenCV for image preprocessing and Histogram of Oriented Gradients (HOG) for feature extraction, while image classification is performed using the K-Nearest Neighbors (KNN) algorithm. OpenMP is utilized to parallelize computationally intensive feature extraction operations to improve execution efficiency and scalability.

B. Problem Statement

Image recognition applications often involve processing large volumes of image data. Sequential image processing approaches execute tasks on a single processor core, resulting in significant computational delays, particularly when handling large datasets. As dataset sizes continue to grow, sequential implementations become increasingly inefficient and inca-

pable of satisfying the performance requirements of real-time applications. Although modern processors provide multiple cores capable of concurrent execution, many conventional image recognition systems do not effectively exploit available multicore resources. Consequently, substantial computational capacity remains underutilized, leading to increased processing time and reduced system efficiency. There is therefore a need to develop efficient parallel image recognition systems capable of leveraging multicore architectures to accelerate computationally intensive operations while maintaining acceptable recognition performance. This project addresses this challenge by designing and implementing a parallel image recognition framework using OpenMP-based multicore programming techniques.

C. Aim of the Study

The aim of this project is to design and implement a parallelized image recognition system using multicore programming techniques in C++ to improve computational performance and scalability.

D. Objectives of the Study

The specific objectives of this project are:

- 1) To develop an image recognition pipeline using C++ and OpenCV.
- 2) To implement image preprocessing techniques for input image normalization.
- 3) To extract discriminative image features using Histogram of Oriented Gradients (HOG).
- 4) To implement image classification using the K-Nearest Neighbors (KNN) algorithm.
- 5) To parallelize computationally intensive operations using OpenMP.
- 6) To evaluate the performance of the proposed system in terms of execution time, speedup, efficiency, and scalability.

E. Significance of the Study

This study demonstrates the practical benefits of multicore programming in accelerating computationally intensive image processing tasks. The findings provide valuable insights into the application of OpenMP for parallel image recognition systems and contribute to understanding how multicore architectures can improve performance in computer vision applications. Furthermore, the project serves as a reference implementation for future research involving parallel image processing and high-performance computing.

F. Scope of the Study

This project focuses on the development of a parallel image recognition system implemented entirely in C++. The system utilizes OpenCV for image preprocessing, HOG descriptors for feature extraction, and KNN for image classification. Parallelization is performed using OpenMP on a shared-memory multicore environment. Experimental evaluation is conducted using a dataset consisting of four image categories: cats,

dogs, airplanes, and automobiles. The study evaluates system performance primarily in terms of execution time, speedup, efficiency, scalability, and classification accuracy.

II. LITERATURE REVIEW

This chapter reviews existing literature related to image recognition systems, multicore computing, OpenMP-based parallel programming, feature extraction techniques, and image classification methods. The chapter further examines previous studies relevant to parallel image recognition systems and identifies existing research gaps addressed by this project.

A. Image Recognition Systems

Image recognition refers to the process of identifying and classifying objects, patterns, or features within digital images. It is a major research area within computer vision and artificial intelligence. Image recognition systems generally consist of several stages, including image acquisition, preprocessing, feature extraction, and classification.

Modern image recognition applications are extensively used in various domains such as autonomous driving, facial recognition, medical diagnosis, industrial inspection, surveillance systems, and robotics. The increasing volume of visual data generated by modern devices necessitates efficient and scalable image processing solutions.

Traditional image recognition systems often process images sequentially, which can lead to increased execution time when handling large image datasets. Consequently, researchers have explored parallel processing approaches to improve computational performance.

B. Multicore Computing

Multicore computing refers to the use of processors containing multiple processing cores within a single computing unit. Each core can execute instructions independently, allowing multiple tasks to run concurrently.

The transition from single-core to multicore processors was primarily motivated by physical limitations associated with increasing processor clock speeds, including power consumption and heat dissipation. Multicore architectures improve system performance by enabling parallel execution rather than relying solely on frequency scaling.

Parallel computing on multicore systems can significantly reduce execution time for computationally intensive applications. However, effective utilization of multicore systems requires the decomposition of tasks into independent units that can be executed simultaneously.

Applications involving image processing are particularly suitable for parallelization because many image operations can be performed independently on different images or image regions.

C. OpenMP-Based Parallel Programming

OpenMP (Open Multi-Processing) is a widely adopted Application Programming Interface (API) that supports shared-memory parallel programming in C, C++, and Fortran.

OpenMP provides compiler directives, runtime library routines, and environment variables that facilitate the development of parallel applications.

One of the primary advantages of OpenMP is its simplicity. Existing sequential programs can often be parallelized by introducing compiler directives such as:

- `pragma omp parallel`
- `pragma omp for`
- `pragma omp sections`

OpenMP automatically manages thread creation, synchronization, workload scheduling, and thread termination, thereby simplifying parallel software development.

Several studies have demonstrated that OpenMP can significantly improve the performance of computationally intensive applications, including image processing, scientific simulations, and machine learning algorithms.

D. Histogram of Oriented Gradients (HOG)

Histogram of Oriented Gradients (HOG) is a feature descriptor introduced by Dalal and Triggs (2005) [3] for object detection tasks. HOG extracts local shape information by computing the distribution of gradient orientations within localized regions of an image.

The HOG algorithm operates through the following steps:

- 1) Gradient computation.
- 2) Division of the image into small cells.
- 3) Construction of orientation histograms.
- 4) Block normalization.
- 5) Feature vector generation.

HOG descriptors are highly effective for capturing edge and shape information while exhibiting robustness against illumination variations and minor geometric transformations.

Due to their computational complexity, HOG computations often represent a significant performance bottleneck in image recognition systems, making them suitable candidates for parallelization.

E. K-Nearest Neighbors (KNN) Classification

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm [4] widely used for classification and pattern recognition tasks. KNN classifies a new sample by identifying the k most similar training instances and assigning the class label based on majority voting.

The algorithm is non-parametric and instance-based, meaning that no explicit training model is generated. Instead, classification relies on distance computations between feature vectors.

Common distance metrics used in KNN include:

- Euclidean Distance
- Manhattan Distance
- Cosine Similarity

Despite its simplicity, KNN can achieve satisfactory classification performance in many image recognition applications. However, the computational cost of distance calculations increases with dataset size, which may affect scalability.

F. Related Work

Numerous researchers have investigated parallel approaches for accelerating image processing and recognition tasks.

Dalal and Triggs (2005) introduced the HOG descriptor [3] for human detection and demonstrated its effectiveness in object recognition tasks. Their work established HOG as one of the most influential handcrafted feature extraction techniques.

Liu et al. (2018) proposed an OpenMP-based parallel image processing framework and reported substantial reductions in execution time when compared to sequential implementations.

Sharma and Gupta (2020) developed a multicore image classification system using OpenMP and observed performance improvements proportional to the number of processing cores employed.

Kim et al. (2021) investigated parallel feature extraction methods for large-scale image datasets and reported improved scalability through shared-memory parallelism.

Although previous studies have demonstrated the effectiveness of multicore approaches, many existing implementations either rely on GPU acceleration or focus on specific image processing tasks. Limited studies have investigated fully C++-based parallel image recognition systems combining HOG feature extraction, KNN classification, and OpenMP-based multicore optimization. This project addresses this gap by developing and evaluating an end-to-end parallel image recognition framework implemented entirely in C++.

G. Research Gap

Existing studies have demonstrated the benefits of parallel processing for image-related applications. However, many implementations depend on GPU architectures, specialized hardware, or deep learning frameworks. Furthermore, limited research has explored lightweight, entirely C++-based image recognition systems utilizing OpenMP for multicore optimization.

This study addresses this gap by developing a complete parallel image recognition system implemented solely in C++, employing OpenCV, HOG feature extraction, KNN classification, and OpenMP-based multicore parallelization.

III. METHODOLOGY

The methodology adopted in the development of the proposed parallel image recognition system. It describes the overall system architecture, dataset acquisition process, image preprocessing procedures, feature extraction techniques, classification approach, multicore parallelization strategy, experimental setup, and performance evaluation metrics employed in this study.

A. Research Methodology

This study adopts an experimental research methodology involving the design, implementation, and evaluation of a parallel image recognition system. The system was developed entirely in C++ and evaluated on a multicore computing environment. OpenCV was employed for image processing operations, while OpenMP was utilized to exploit multicore

TABLE I
SUMMARY OF RELATED STUDIES

Author	Methodology	Parallel Technique	Key Findings	Limitations
Dalal & Triggs (2005)	HOG-based object detection [?]	None	Introduced the HOG descriptor with high detection accuracy	Computationally expensive
Liu <i>et al.</i> (2018)	Image processing framework	OpenMP	Reduced execution time significantly	Limited scalability analysis
Sharma & Gupta (2020)	Parallel image classification	OpenMP	Achieved improved classification performance	Evaluated on small datasets
Kim <i>et al.</i> (2021)	Parallel feature extraction	Shared-memory parallelism	Improved scalability and throughput	Did not evaluate classification performance
Proposed Study	HOG + KNN image recognition	OpenMP	Improved execution time and scalability	Moderate classification accuracy

capabilities for performance optimization. The methodology consists of the following stages:

- 1) Dataset acquisition.
- 2) Image preprocessing.
- 3) Feature extraction.
- 4) Image classification.
- 5) Parallelization using OpenMP.
- 6) Performance evaluation.

B. System Architecture

The proposed image recognition framework consists of multiple interconnected modules responsible for processing images and evaluating system performance. The major system components include:

- Image Loader.
- Image Preprocessor.
- Feature Extractor.
- K-Nearest Neighbors Classifier.
- OpenMP Parallel Processing Module.
- Performance Evaluation Module.

The overall workflow of the proposed system is illustrated in Figure 1.

The processing pipeline begins with loading images from the dataset. Images subsequently undergo preprocessing operations before HOG descriptors are extracted. Feature vectors are then classified using the KNN algorithm. Computationally intensive operations are parallelized using OpenMP to improve execution efficiency.

C. Dataset Acquisition

The dataset employed in this study consists of four image categories:

- Cats
- Dogs
- Airplanes
- Automobiles

A total of 4020 images were collected and organized into separate class folders. Images were obtained from publicly available online image repositories and arranged according to their respective categories. Table ?? presents the dataset distribution.

TABLE II
DISTRIBUTION OF IMAGES IN THE DATASET

Class	Number of Images
Cat	1005
Dog	1005
Airplane	1005
Automobile	1005
Total	4020

The dataset directory structure is shown below:

```
dataset/
|-- cat/
|-- dog/
|-- airplane/
`-- automobile/
```

Each folder contains images corresponding to its respective class label.

D. Image Preprocessing

Image preprocessing was performed to normalize image dimensions and improve consistency prior to feature extraction. The preprocessing stage involved the following operations.

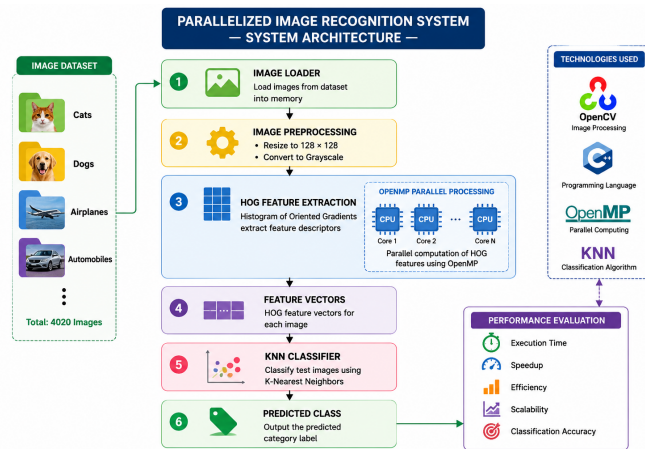


Fig. 1. Overall architecture of the proposed parallel image recognition system.

1) *Image Resizing*: All images were resized to a uniform dimension of 128×128 pixels. Standardizing image dimensions ensures consistent feature extraction and reduces computational complexity.

E. Grayscale Conversion

Images were converted from RGB color space to grayscale format. Grayscale conversion reduces memory requirements and computational overhead while preserving essential structural information required for HOG feature extraction. The grayscale conversion operation is represented mathematically as:

$$I_{gray} = 0.299R + 0.587G + 0.114B \quad (1)$$

where:

- R represents the red channel.
- G represents the green channel.
- B represents the blue channel.

F. Feature Extraction Using HOG

Feature extraction was performed using Histogram of Oriented Gradients (HOG) descriptors. HOG extracts local edge information by computing gradient orientations within localized image regions. The HOG configuration used in this project includes:

- Window Size: 128×128
- Block Size: 16×16
- Block Stride: 8×8
- Cell Size: 8×8
- Number of Orientation Bins: 9

The HOG extraction procedure consists of:

- 1) Gradient computation.
- 2) Cell histogram generation.
- 3) Block normalization.
- 4) Feature vector construction.

The resulting feature vectors serve as input to the classifier.

G. Image Classification Using KNN

Image classification was performed using the K-Nearest Neighbors (KNN) algorithm. KNN classifies unknown samples by comparing feature vectors against previously labeled training samples. The classifier uses Euclidean distance to determine similarity between feature vectors. The Euclidean distance metric is given by:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (2)$$

where:

- x represents the feature vector of the test image.
- y represents the feature vector of a training image.

The class label receiving the majority vote among the nearest neighbors is assigned to the test sample. An 80:20 train-test split was employed during experimental evaluation.

H. OpenMP-Based Parallelization Strategy

To exploit multicore architectures, computationally intensive operations were parallelized using OpenMP. Feature extraction was identified as the most time-consuming component of the image recognition pipeline. Parallelization was implemented using the OpenMP directive:

```
#pragma omp parallel for
```

Each thread independently processes a subset of images, thereby enabling concurrent execution across multiple processor cores. The parallel implementation distributes HOG computations among available threads, significantly reducing execution time. Figure 2 illustrates the parallel execution model.

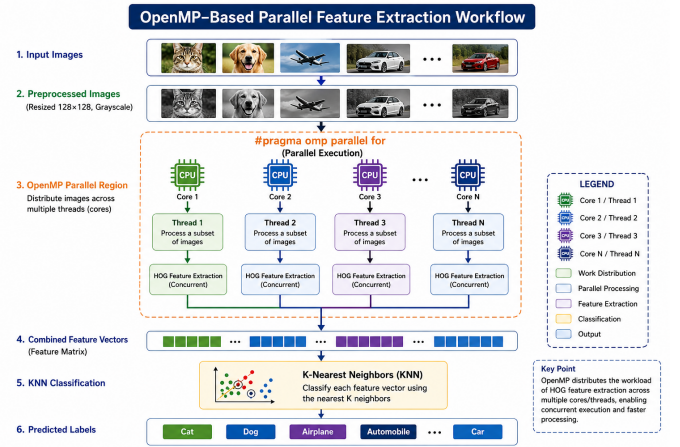


Fig. 2. OpenMP-based parallel feature extraction workflow.

I. Experimental Setup

Experimental evaluation was conducted on the hardware and software configuration shown in Table III.

TABLE III
EXPERIMENTAL ENVIRONMENT

Component	Specification
Processor	Apple M1
Operating System	macOS Tahoe
Programming Language	C++
Development Environment	Visual Studio Code
Parallel Framework	OpenMP
Computer Vision Library	OpenCV 4.13
Dataset Size	4020 Images

The experiments were performed using varying thread counts:

- 1 Thread
- 2 Threads
- 4 Threads
- 8 Threads

J. Performance Evaluation Metrics

The proposed system was evaluated using the following performance metrics.

1) *Execution Time*: Execution time measures the total time required to perform feature extraction.

2) *Speedup*: Speedup quantifies performance improvement achieved through parallelization.

$$Speedup = \frac{T_{sequential}}{T_{parallel}} \quad (3)$$

where:

- $T_{sequential}$ represents sequential execution time.
- $T_{parallel}$ represents parallel execution time.

K. Efficiency

Efficiency evaluates processor utilization effectiveness.

$$Efficiency = \frac{Speedup}{Number\ of\ Threads} \quad (4)$$

L. Classification Accuracy

Classification accuracy measures the percentage of correctly classified images.

$$Accuracy = \frac{Correct\ Predictions}{Total\ Predictions} \times 100 \quad (5)$$

IV. RESULTS AND DISCUSSION

The experimental results obtained from evaluating the proposed parallel image recognition system. The system was assessed in terms of execution time, speedup, efficiency, scalability, and classification performance. Experimental evaluations were conducted using different numbers of processing threads to investigate the impact of multicore parallelization on overall system performance.

A. Experimental Results

Experiments were performed using four different thread configurations:

- 1 Thread
- 2 Threads
- 4 Threads
- 8 Threads

The objective was to compare the performance of sequential and parallel feature extraction implementations under varying levels of parallelism. Table IV summarizes the experimental results obtained.

TABLE IV
PERFORMANCE EVALUATION RESULTS

Threads	Sequential Time (s)	Parallel Time (s)	Speedup	Efficiency (%)
1	0.977	0.970	1.008	100.75
2	0.999	0.530	1.883	94.17
4	0.988	0.281	3.518	87.96
8	0.978	0.216	4.536	56.70

The results demonstrate that increasing the number of threads significantly reduces execution time while improving overall system performance.

B. Execution Time Analysis

Figure 3 illustrates the relationship between the number of threads and parallel execution time.

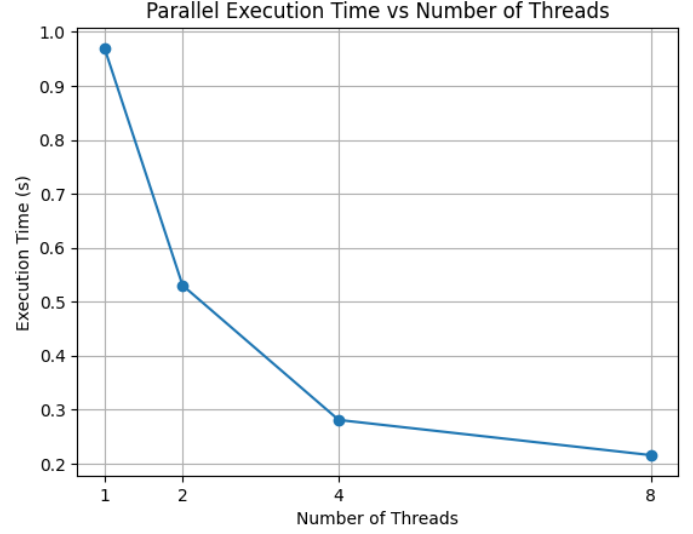


Fig. 3. Parallel execution time versus number of threads.

The results indicate a substantial reduction in execution time as additional threads are employed. The execution time decreased from approximately 0.97 seconds when using a single thread to approximately 0.22 seconds when using eight threads. This improvement confirms the effectiveness of multicore parallelization in accelerating computationally intensive feature extraction operations.

C. Speedup Analysis

Figure 4 presents the speedup achieved by the parallel implementation under different thread configurations.

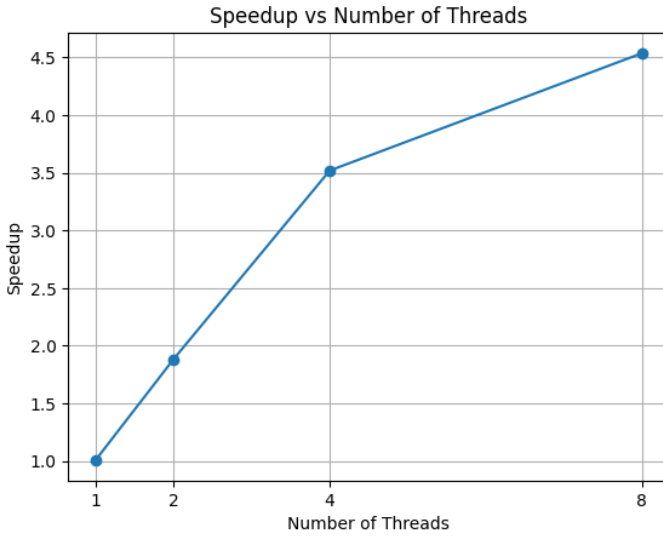


Fig. 4. Speedup versus number of threads.

The speedup increased consistently with the number of processing threads. The maximum speedup achieved was approximately $4.54\times$ when eight threads were employed. This result demonstrates that the proposed OpenMP implementation successfully exploits multicore resources to accelerate image feature extraction. Although the observed speedup is not perfectly linear, the results clearly indicate significant performance gains compared to sequential execution.

D. Efficiency Analysis

Figure 5 illustrates processor efficiency under varying thread counts.

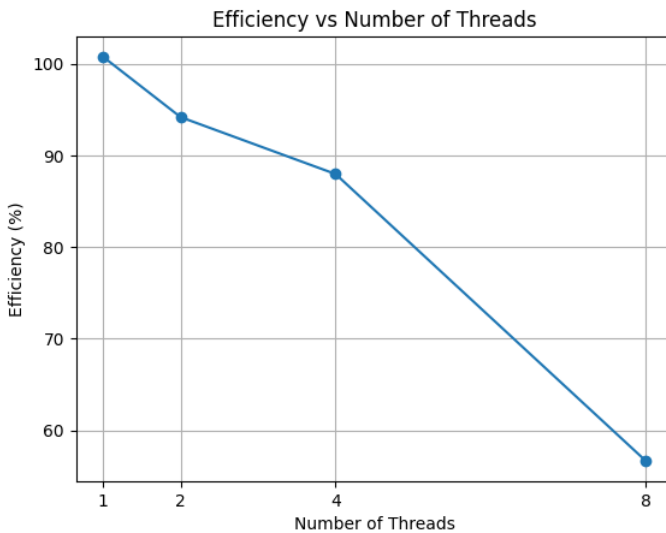


Fig. 5. Efficiency versus number of threads.

The highest efficiency values were observed when using one, two, and four threads. Efficiency gradually decreased

as the number of threads increased, reaching approximately 56.7% for eight threads. This reduction in efficiency can be attributed to several factors, including:

- Thread management overhead.
- Synchronization costs.
- Memory contention among concurrent threads.
- Load imbalance across processing cores.

Despite the reduction, the efficiency values remain acceptable for a shared-memory multicore environment.

E. Sequential and Parallel Comparison

Figure 6 compares sequential and parallel execution times.

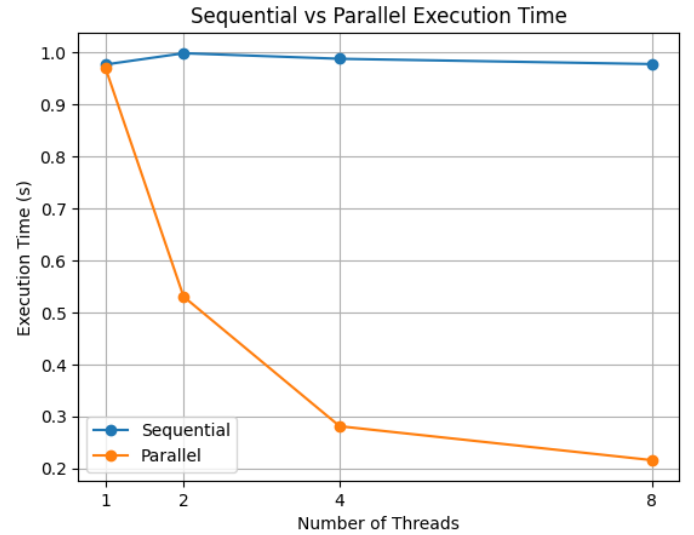


Fig. 6. Sequential versus parallel execution time comparison.

The comparison clearly demonstrates that the parallel implementation consistently outperforms the sequential implementation across all thread configurations. As thread count increases, the gap between sequential and parallel execution times becomes increasingly pronounced. The experimental results validate the effectiveness of the proposed multicore approach for improving image processing performance.

F. Classification Performance

The proposed system was further evaluated in terms of classification performance. The dataset was divided into training and testing subsets using an 80:20 split ratio. Experimental evaluation produced an overall classification accuracy of approximately 40.92

TABLE V
CLASSIFICATION PERFORMANCE

Metric	Value
Testing Samples	804
Correct Predictions	329
Classification Accuracy	40.92%

Although the achieved accuracy is moderate, it is important to note that the primary objective of this project was to investigate multicore parallelization rather than maximizing classification accuracy. The use of HOG descriptors combined with K-Nearest Neighbors classification provided sufficient functionality for evaluating performance improvements introduced through parallel computing.

DISCUSSION

The experimental findings demonstrate that multicore programming significantly improves the performance of image recognition systems. Parallel feature extraction using OpenMP reduced execution time considerably while achieving substantial speedup. The maximum observed speedup of $4.54\times$ confirms that multicore architectures can effectively accelerate computationally intensive image processing operations. The decline in efficiency observed at higher thread counts is consistent with established parallel computing principles, where synchronization overhead and resource contention limit scalability. Overall, the proposed system successfully achieved its objectives by:

- 1) Reducing execution time.
- 2) Improving computational throughput.
- 3) Exploiting multicore processor resources.
- 4) Demonstrating scalable parallel performance.

These findings validate the suitability of OpenMP as a lightweight and effective framework for multicore image processing applications.

V. CONCLUSION AND FUTURE WORK

This project presented the design and implementation of a Parallelized Image Recognition System using multicore programming techniques. The system was developed entirely in C++ using OpenCV for image preprocessing and feature extraction, K-Nearest Neighbors (KNN) for image classification, and OpenMP for parallel processing.

The study addressed the limitations associated with sequential image processing by exploiting multicore processor architectures to accelerate computationally intensive tasks. The proposed system successfully implemented a complete image recognition pipeline consisting of image loading, preprocessing, Histogram of Oriented Gradients (HOG) feature extraction, classification, and performance evaluation.

Experimental evaluation was conducted using a dataset containing 4020 images distributed across four classes: cats, dogs, airplanes, and automobiles. Performance analysis demonstrated that parallel feature extraction significantly reduced

execution time compared to the sequential implementation. The proposed system achieved a maximum speedup of approximately $4.54\times$ when executed using eight processing threads.

The results further revealed that execution time decreased consistently as the number of processing threads increased. Although efficiency decreased at higher thread counts due to synchronization overhead and resource contention, the overall performance improvements confirmed the effectiveness of multicore parallelization.

Furthermore, the study demonstrated that OpenMP provides a lightweight and efficient framework for implementing shared-memory parallelism in image processing applications. The findings validate the potential of multicore programming for improving computational performance in computer vision systems.

Overall, the objectives of this project were successfully achieved. The developed system effectively exploited multicore architectures to accelerate image recognition tasks while maintaining functional classification performance.

A. Contributions of the Study

The major contributions of this project include:

- 1) The design and implementation of a complete image recognition framework using C++.
- 2) The development of a multicore parallel feature extraction mechanism using OpenMP.
- 3) The integration of HOG feature extraction and KNN classification within a unified image recognition pipeline.
- 4) The experimental evaluation of multicore performance using execution time, speedup, efficiency, and scalability metrics.
- 5) The demonstration of the effectiveness of OpenMP-based parallelization for computationally intensive image processing tasks.

B. Limitations of the Study

Despite the successful implementation of the proposed system, several limitations were identified:

- 1) The classification accuracy achieved was moderate due to the use of handcrafted HOG features and a simple KNN classifier.
- 2) The study focused exclusively on CPU-based multicore architectures and did not consider GPU acceleration.
- 3) The experiments were conducted on a single hardware platform, which may limit the generalizability of the performance results.
- 4) The system was evaluated using only four image classes.
- 5) Deep learning-based feature extraction methods were not investigated.

C. Future Work

Future research can extend the present study in several directions:

- 1) Integrate deep learning techniques such as Convolutional Neural Networks (CNNs) to improve classification accuracy.
- 2) Investigate GPU-based parallelization using CUDA or OpenCL to further accelerate image processing operations.
- 3) Evaluate the proposed framework on larger and more diverse image datasets.
- 4) Explore hybrid CPU-GPU parallel architectures for improved scalability.
- 5) Implement distributed image processing techniques using MPI to support large-scale image recognition applications.
- 6) Investigate advanced classification algorithms such as Support Vector Machines (SVMs) and Random Forests.
- 7) Extend the framework to support real-time image recognition applications.

D. Final Remarks

The rapid growth of visual data continues to create significant computational challenges for image processing systems. This project has demonstrated that multicore programming provides an effective and practical solution for improving the performance of image recognition applications. The successful implementation and evaluation of the proposed system provide a strong foundation for future research in high-performance image processing and parallel computer vision systems.

REFERENCES

- [1] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.
- [2] OpenMP Architecture Review Board, “Openmp api specification,” <https://www.openmp.org/specifications/>, 2026.
- [3] N. Dalal and B. Triggs, “Histograms of oriented gradients for human detection,” in *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005, pp. 886–893.
- [4] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, 2009.